Pésentation JEE

Mohammed OUANAN

m.ouanan@umi.ac.ma

Plan

1. Introduction
2. Fonctionnement
3. Installation et configuration
   - JDK
   - IDE
   - Serveur HTTP
4. Création d'un premier projet web (avec Eclipse)
5. Structure d'un projet JEE réalisé sous Eclipse
6. Premier Hello world

Le besoin de JEE?

- Dynamiser le contenu web
  - Apparition de génération dynamique de code (ASP,-1996- 2002, PHP-1997 , JEE - 1999)
  - Apparition des applets (exécution code java sur navigateur)
- Echanger des informations entre applications
  - RMI, Corba, EJB
- Création de véritables d'architectures logicielles
  - Architectures 3tiers, Modèle-Vue-Contrôleur
- Harmonisation des infrastructures

Jakarta EE

JEE

- 1999 : inclus dans **Java 2** sous le nom **J2EE** pour **J**ava 2 Platform **E**nterprise **E**dition
- 2006 : renommé depuis **Java 5** en **JEE** pour **J**ava Platform **E**nterprise **E**dition
- 2017 : **Oracle** confie le projet à **Eclipse Foundation** qui décide de le renommer **Jakarta EE**

Jakarta EE

JEE

- Plateforme facilitant le développement d'applications d'entreprise distribuées
- Développée par **Sun** puis **Oracle**
- **JEE** = **JSE** + plusieurs autres **API**
  - **JSE** : **J**ava **S**tandard **E**dition (anciennement J2SE)
  - **API** : Application Programming Interface

Les API de JEE?

Java EE (aujourd'hui appelé **Jakarta EE**) repose sur deux grands éléments :

Les spécifications de l'infrastructure

Elles décrivent comment doit fonctionner l'environnement d'exécution qui héberge les applications Java. Cet environnement est appelé **serveur d'applications** (par exemple : WildFly, GlassFish, JBoss, WebLogic…).Il fournit tous les services nécessaires (gestion des transactions, sécurité, accès aux bases de données, communication distribuée, etc.) pour exécuter correctement les composants.

Les API (Interfaces de Programmation d'Applications)

Ce sont des bibliothèques standardisées mises à disposition des développeurs. Elles couvrent plusieurs besoins comme : JDBC (accès aux bases de données), Servlets/JSP (interfaces web), JPA (persistance des données), JMS (messagerie), etc. Ces API peuvent être utilisées **indépendamment** du serveur d'application, par exemple dans une simple application Java SE.

API Jakarta EE

Les API de Java EE peuvent se répartir en deux grandes catégories :

- Les composants déployés sur le serveur
  - Les composants web qui sont réalisés à l'aide de servlets, de JavaServer Pages (JSP) ou de Java server face (JSF).
  - Les composants métier sont des Entreprise JavaBeans (EJB). Il s'agit de composants spécifiques chargés des traitements des données et de l'interfaçage avec les bases de données.
- Les services
  - Les services d'infrastructures : JDBC, JNDI, JTA, JCA, JMX
  - Les services de communication : RMI-IIOP, JavaMail, JAAS

Les principaux composants API Jakarta EE

- **Servlet**: Utilise les mécanismes de requetes-reponses http afin d'effectuer des tâches ou de générer des pages html
- **EJB** : définit la façon dont les composants doivent être écrits et le contrat qu'ils doivent respecter avec le serveur d'application
- **RMI** : communication inter procédés
- **JNDI** : C'une API d'accès aux services de nommage et aux annuaires d'entreprises tels que DNS, NIS, LDAP, etc.
- **JDBC** : connexion vers les bases de données
- **JTA** : service de transaction
- **JMS** : service de messagerie
- **JSP** : Java Server Page
- **Java IDL** : intégration aux autres langages (ex: CORBA) JavaMail
- **Connectors** : intégration à des middlewares existants XML

Architecture JEE

J2EE permet une grande flexibilité dans le choix de l'architecture de l'application en combinant les différents composants. Ce choix dépend des besoins auxquels doit répondre l'application mais aussi des compétences dans les différentes API de J2EE. D'après le schéma ci-après, la spécification Java EE définit les composants suivants :

- Les composants Clients (ou tier Client) s'exécutent sur la machine des utilisateurs ;
- Les composants Web(ou tiers Web) s'exécutent sur le serveur J2EE ;
- Les composants métier ou (tier Métier) s'exécutent sur le serveur Java EE ;
- La partie stockage des données, nommée tiers Enterprise Information System (EIS), s'exécute sur le serveur de base de données

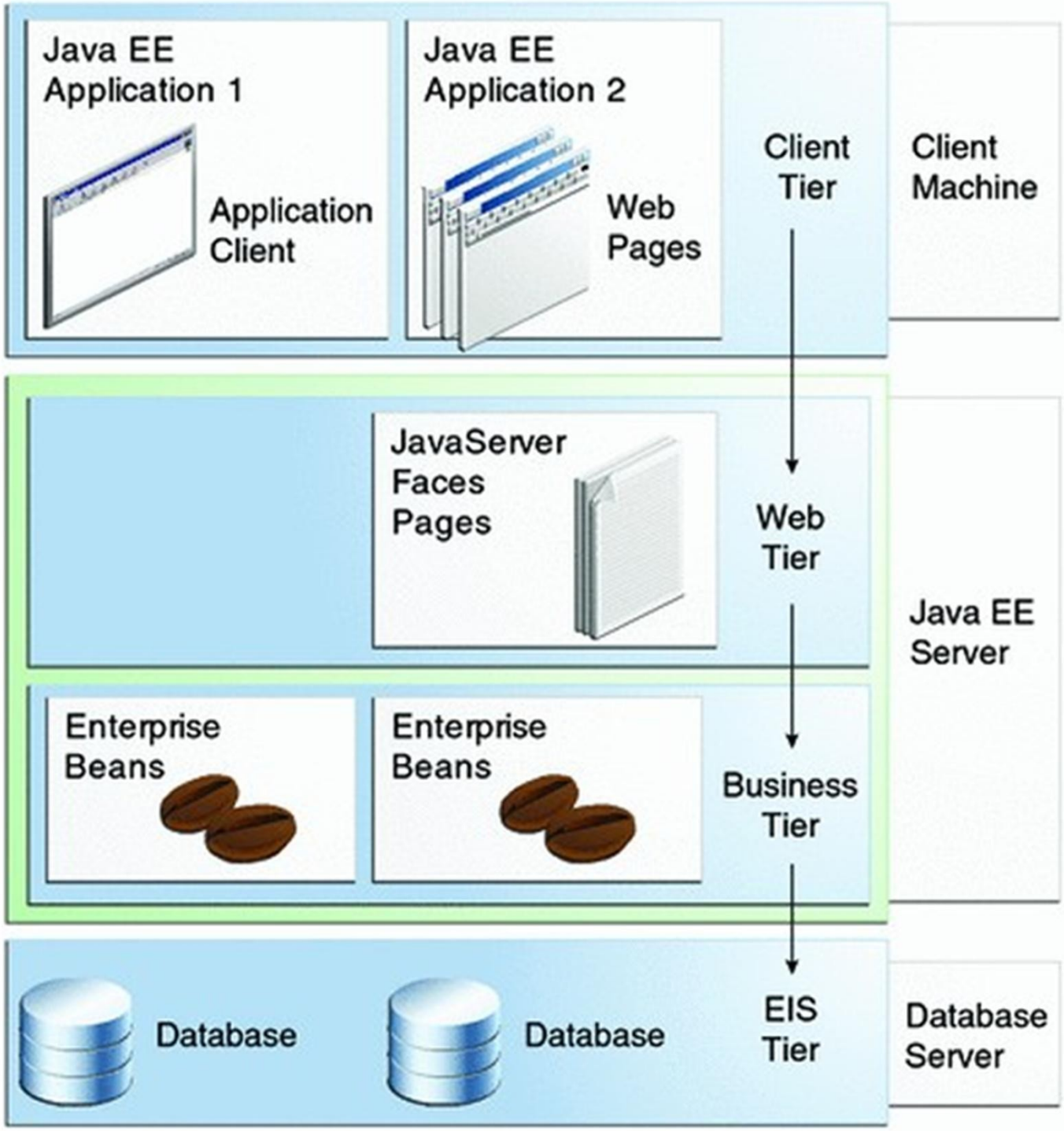


Figure Architecture Java EE standard

Jakarta EE côté client

- Un client Java EE peut être :
  - une application console écrite en Java
  - une application dotée d'une interface graphique Swing.

  Ce type de client est appelé client lourd, en raison de la quantité importante de code qu'il met en œuvre.
- Un client Java EE peut également être conçu pour être utilisé à partir du Web.
  - Ce type de client fonctionne à l'intérieur d'un navigateur Web.
  - La plus grande partie du travail est reportée sur le serveur et le client ne comporte que très peu de code. Pour cette raison, on parle de client léger.

Un client léger peut être une simple interface HTML, une page contenant des scripts JavaScript, ou encore une applet Java si une interface un peu plus riche est nécessaire.

Les conteneurs JEE

Un conteneur (container) est l'interface entre le composant et les services de bas niveaux nécessaires: assure la gestion du cycle de vie des composants qui s'exécutent en eux et fournissent des services qui peuvent être utilisés par les applications lors de leur exécution

Le serveur Java EE est composé de deux types de conteneurs :

- Web Container ou conteneur Web, capable d'exécuter des pages JSP, Servlets et programmes JSF (implémentation JSF nécessaire). Ces composants s'exécutent avec le conteneur serveur Java EE. Ce conteneur supporte les protocoles HTTP et HTTPS et dialogue avec différents types de clients (navigateur, application) ;
- EJB Container ou conteneur EJB, responsable de la couche métier et de l'exécution des EJB. Ce conteneur fournit le mécanisme de gestion des EJB et la gestion de leur cycle de vie ;

L'assemblage et le déploiement d'applications J2EE

Une application Jakarta EE est donc composée d'un ensemble d'unités de programmation qui pourront ensuite être déployées sur n'importe quelle plate-forme compatible avec les spécifications Jakarta EE.

Chaque unité contient :

- **Un ou plusieurs composants fonctionnels (ejb, pages JSP, servlet, etc.)**
- **Des descripteurs de déploiement (optionnels) qui spécifient le contenu du composant.**

On parle ainsi du packaging d'une application

Les fichiers EAR

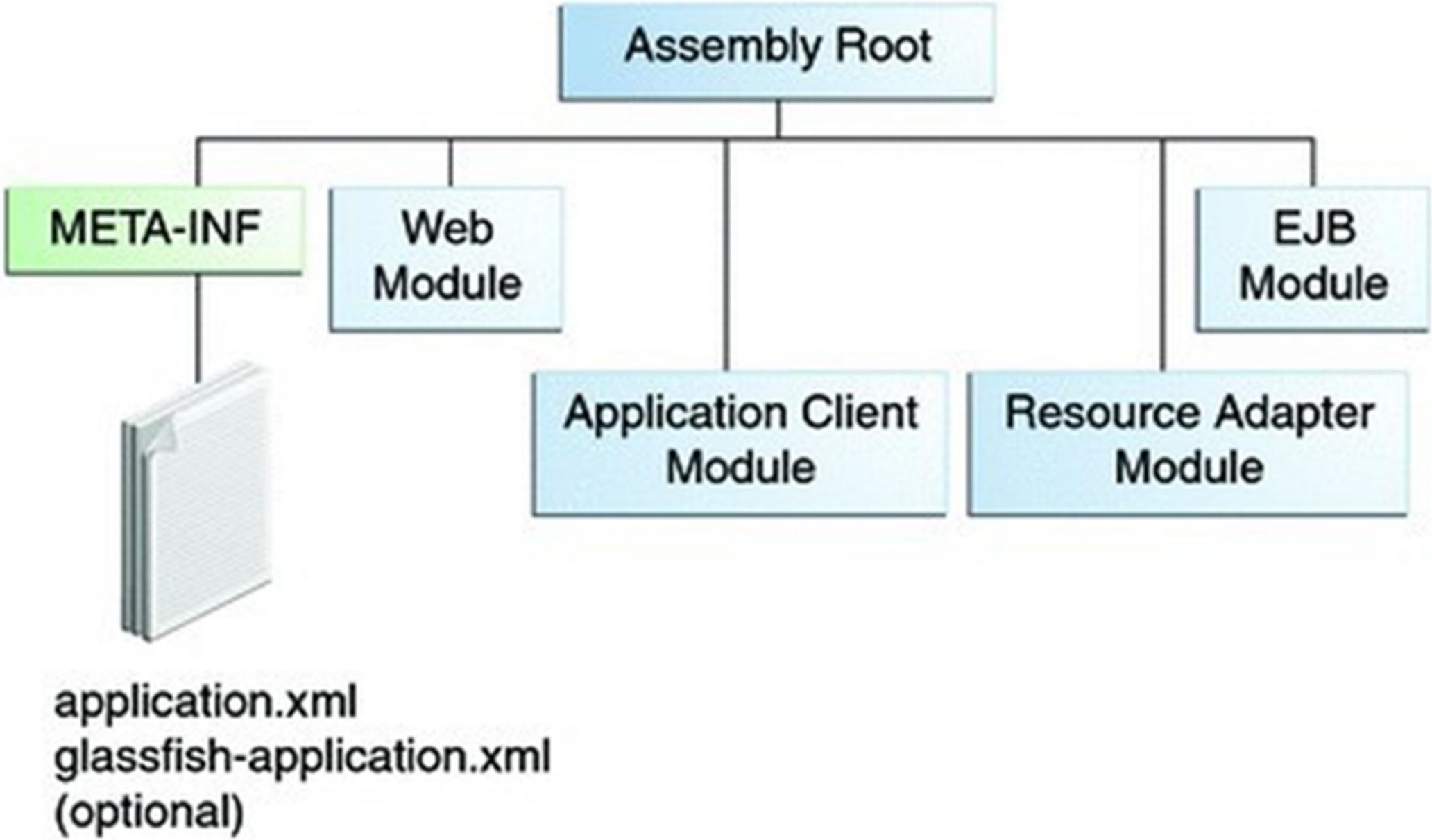
Une application Jakarta EE est distribuée sous forme d'un fichier *Enterprise Archive* (EAR) C'est en fait un simple fichier jar avec l'extension .ear

Ce fichier contient :

- des modules Jakarta EE (.jar, .war, .rar)
- Les descripteurs de déploiement (des fichiers xml). Ils peuvent donc être modifiés sans toucher le code de l'application.

À l'exécution, le serveur lit les descripteurs pour utiliser les composants de manière adéquate.

Les fichiers EAR



Archive / module	Contenu	Extension	Descripteur de déploiement
application client	Regroupe les ressources nécessaires à leur exécution (classes, bibliothèques, images, ...)	jar	application-client.xml
Web	Regroupe les servlets et les JSP ainsi que les ressources nécessaires à leur exécution (classes, bibliothèques de balises, images, ...)	war	web.xml
EJB	Regroupe les EJB et leurs composants (classes)	jar	ejb-jar.xml

Les différents types d'archives

Comparaison des fichiers JAR, WAR et EAR

sont des formats d'archives utilisés pour regrouper et distribuer des applications. Chacun de ces formats a un objectif spécifique, lié à l'environnement dans lequel l'application sera déployée (standalone ou serveur d'applications)

Type	Contenu	Utilisation principale	Environnement de déploiement
JAR	Classes Java, bibliothèques, métadonnées	Applications autonomes ou bibliothèques Java	JVM standalone
WAR	Application web, fichiers JSP/HTML, classes	Applications web Java (Servlets, JSP, Spring)	Serveur d'application web (Tomcat, Jetty)
EAR	Modules JAR et WAR, fichiers de configuration	Applications d'entreprise Java EE (EJB, Web) complexes	Serveur d'application Java EE (WildFly, GlassFish)

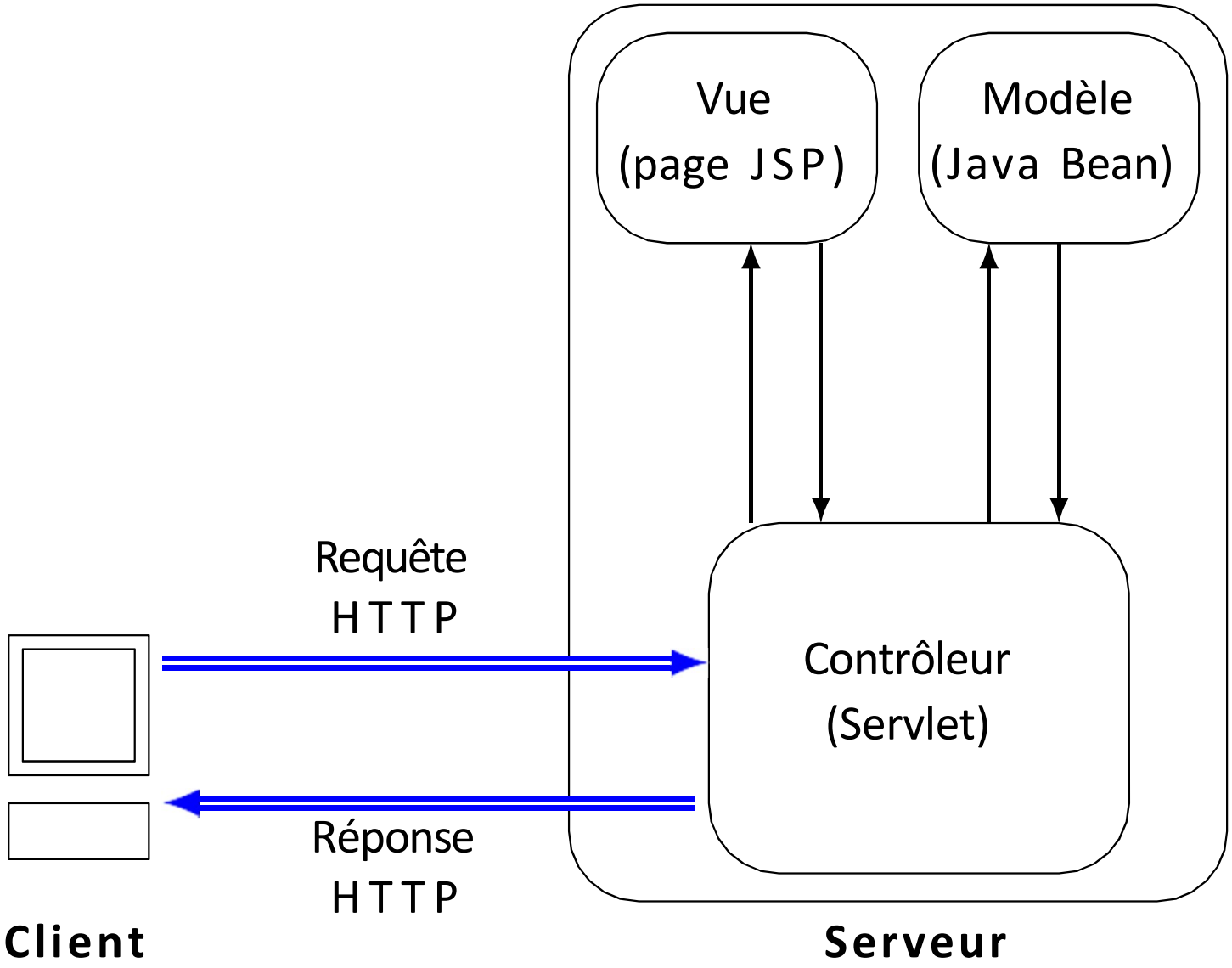
Les descripteurs

Il existe 2 types de descripteurs :

- Les Jakarta EE **deployment descriptor** : peuvent être utilisés pour configurer les paramètres de déploiement du module
- Les **runtime deployment descriptor** : utilisé pour paramétrer l'environnement hôte, c'est-à-dire n'importe quelle plate-forme compatible avec la norme Jakarta EE.
- Par exemple, pour la plate-forme *Sun Java System Application Server Platform Edition 9*, le runtime descriptor définit des informations comme la racine du répertoire web.
- Les noms de ces fichiers sont standards :

  sun-moduleType**.xml** (dans le répertoire META-INF)

Jakarta EE


Vue
(page JSP)
Modèle
(Java Bean)
Requête
HTTP
Contrôleur
(Servlet)
Réponse
HTTP
Client
Serveur

Jakarta EE

Déroulement

- L'échange entre le client et le serveur s'effectue via le modèle **HTTP** (requête - réponse)
- Lorsque l'utilisateur saisit l'adresse d'une page de notre site, cette dernière sera envoyée sous forme de requête **HTTP** au contrôleur
- Le contrôleur demande au modèle de lui fournir certaines données
- Ensuite il renvoie ces données à la vue pour qu'elle construise la page **HTML**
- Enfin le client reçoit la réponse sous forme de page **JSP**

Jakarta EE

De quoi on a besoin

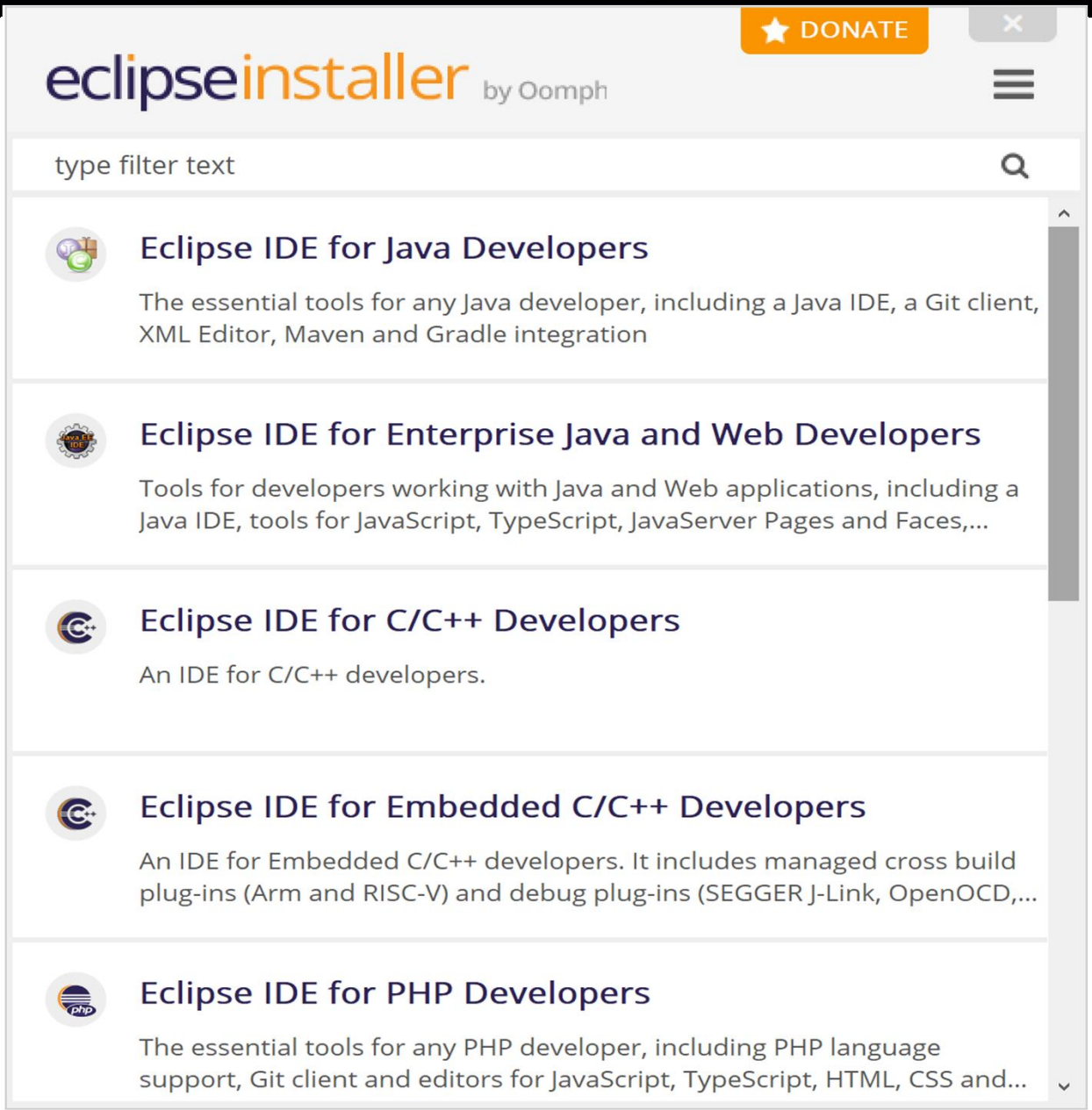
- **JDK** : **J**ava **D**evelopment **K**it
- **IDE** : **I**ntegrated **D**evelopment **E**nvironment (Environnement de Développement Intégré)
- Un serveur **HTTP**

Jakarta EE

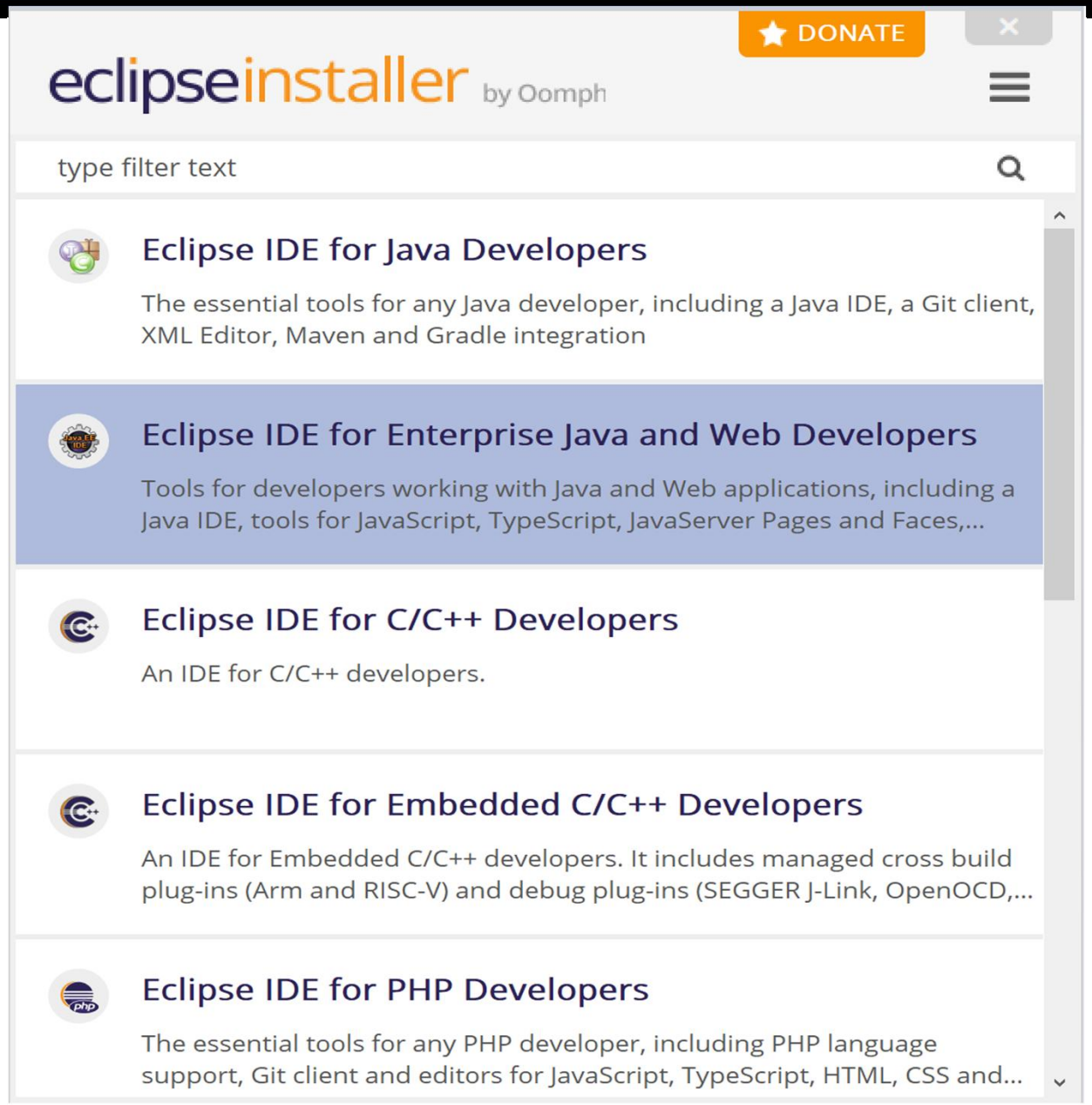
Environnement de Dévloppement Intégré

- **Eclipse**
- Netbeans
- JDeveloper
- IntelliJ IDEA
- JBuilder
- JCreator...
- ...

Jakarta EE


DONATE
eclipseinstaller by Oomph
type filter text
Eclipse IDE for Java Developers
The essential tools for any Java developer, including a Java IDE, a Git client, XML Editor, Maven and Gradle integration
Eclipse IDE for Enterprise Java and Web Developers
Tools for developers working with Java and Web applications, including a Java IDE, tools for JavaScript, TypeScript, JavaServer Pages and Faces,...
Eclipse IDE for C/C++ Developers
An IDE for C/C++ developers.
Eclipse IDE for Embedded C/C++ Developers
An IDE for Embedded C/C++ developers. It includes managed cross build plug-ins (Arm and RISC-V) and debug plug-ins (SEGGER J-Link, OpenOCD,...
Eclipse IDE for PHP Developers
The essential tools for any PHP developer, including PHP language support, Git client and editors for JavaScript, TypeScript, HTML, CSS and...

Jakarta EE


DONATE
eclipseinstaller by Oomph
type filter text
Eclipse IDE for Java Developers
The essential tools for any Java developer, including a Java IDE, a Git client, XML Editor, Maven and Gradle integration
Eclipse IDE for Enterprise Java and Web Developers
Tools for developers working with Java and Web applications, including a Java IDE, tools for JavaScript, TypeScript, JavaServer Pages and Faces,...
Eclipse IDE for C/C++ Developers
An IDE for C/C++ developers.
Eclipse IDE for Embedded C/C++ Developers
An IDE for Embedded C/C++ developers. It includes managed cross build plug-ins (Arm and RISC-V) and debug plug-ins (SEGGER J-Link, OpenOCD,...
Eclipse IDE for PHP Developers
The essential tools for any PHP developer, including PHP language support, Git client and editors for JavaScript, TypeScript, HTML, CSS and...

Jakarta EE

Modifier l'encodage par défaut

- Aller dans le menu `Window` et cliquer sur `Preferences`
- Dans la zone de recherche, écrire `encoding`
- Pour toutes les sections, mettre `UTF-8` à la place de l'encodage par défaut (généralement Cp1252 ou ISO-8859-1)
- Ensuite appliquer et fermer

Java

Quelques raccourcis **Eclipse**

- Ctrl + Shift + : : commenter/décommenter le code
- Ctrl + Shift + f : formater le code
- Ctrl + Alt + ↓ ou Ctrl + Alt + ψ : dupliquer la ligne sélectionnée
- Ctrl + Shift + o : gérer les importer
- Ctrl + Alt + l : afficher la liste des raccourcis
- Alt + Shift + R : faire une sélection multiple
- Shift + K : aller à l'occurrence suivante
- Ctrl + Shift + K : aller à l'occurrence précédente

Serveur **HTTP**

- **Apache Tomcat**
- WebLogic Server (Serveur payant d'oracle utilisé par JDeveloper)
- JBoss
- GlassFish (Open Source de Oracle)
- ...

Serveur **HTTP**

- **Apache Tomcat**
- WebLogic Server (Serveur payant d'oracle utilisé par JDeveloper)
- JBoss
- GlassFish (Open Source de Oracle)
- ...

Pourquoi **Apache Tomcat** ?

- gratuit
- multi-plateforme
- léger
- ...

Jakarta EE

Mise en place d'**Apache Tomcat**

- Pour télécharger, aller sur https://tomcat.apache.org/download-10.cgi
- Aller dans la section `Core`
- Cliquer sur `32-bit/64-bit Windows Service Installer`
- Attendre la fin du téléchargement puis lancer l'installation
- Installer **Apache Tomcat** à la racine de votre disque dur (`C:` pour Windows) et vérifier que le nom du dossier destination ne contient pas d'espace
- Décocher la case de la dernière fenêtre d'installation proposant de démarrer **Apache Tomcat**

Jakarta EE

Les étapes

- Aller dans le menu `File > New` et cliquer sur `Other` (ou CTRL + N)
- Ensuite choisir `Dynamic Web Project` situé dans le répertoire `Web`
- Saisir `cours-jee` dans `Project name`
- Ensuite cliquer sur le bouton `New Runtime`, choisir la dernière version d'Apache Tomcat et cocher la case juste en-dessous `create a new local server`
- Cliquer sur `Next` et préciser le répertoire d'installation de **Apache Tomcat**
- Cliquer sur `Finish` ensuite deux fois sur `Next`, puis cocher la case `Generate web.xml deployment descriptor`

Jakarta EE

Les étapes

- Faire un clic droit sur le projet et choisir `Run As` ensuite `Run on Server`
- Sélectionner le serveur **Apache Tomcat** et cocher la case `Always use this server when running this project`
- Cliquer sur `Next` et vérifier que notre projet figure dans la liste `Configured`
- Et enfin valider en cliquant sur `Finish`

Jakarta EE

Structure d'une application **JEE**

- Le nom du projet définit la racine de l'arborescence
- La racine contient principalement trois répertoires
  - `Java Resources` est réservé aux classes **Java**
  - `src` contient un dossier `main` contenant deux répertoires `java` et `webapp`
    - `java` pointe vers `Java Resources`
    - `webapp` contient tous les fichiers web
  - `build` contient les fichiers compilés (d'extension `.class`)

Jakarta EE

Contenu de `webapp`

Les fichiers définis directement dans `webapp` sont accessibles aux visiteurs (sans avoir besoin de passer par une **Servlet**). C'est ici qu'on place les fichiers **CSS**, **JavaScript**...

- `WEB-INF` (inaccessible aux visiteurs) a un dossier `lib` (pour les librairies externes d'extension `.jar`) et un fichier `web.xml` (premier fichier consulte´ par le serveur **HTTP**).
- `META-INF` contient les méta-données du projet.

Jakarta EE

Création d'une page **HTML**

- Clic droit sur le projet, aller dans le menu `new` et choisir `HTML File`
- Placer le fichier directement dans `webapp` (pas dans `WEB-INF`)
- Nommer le fichier `index.html` et générer la page en cliquant sur Finish
- Mettre `Hello world` entre les balises `<body>`
- Démarrer le serveur
- Aller à l'adresse `http://localhost:8080/cours-jee/` ou `http://localhost:8080/cours-jee/index.html`

Jakarta EE

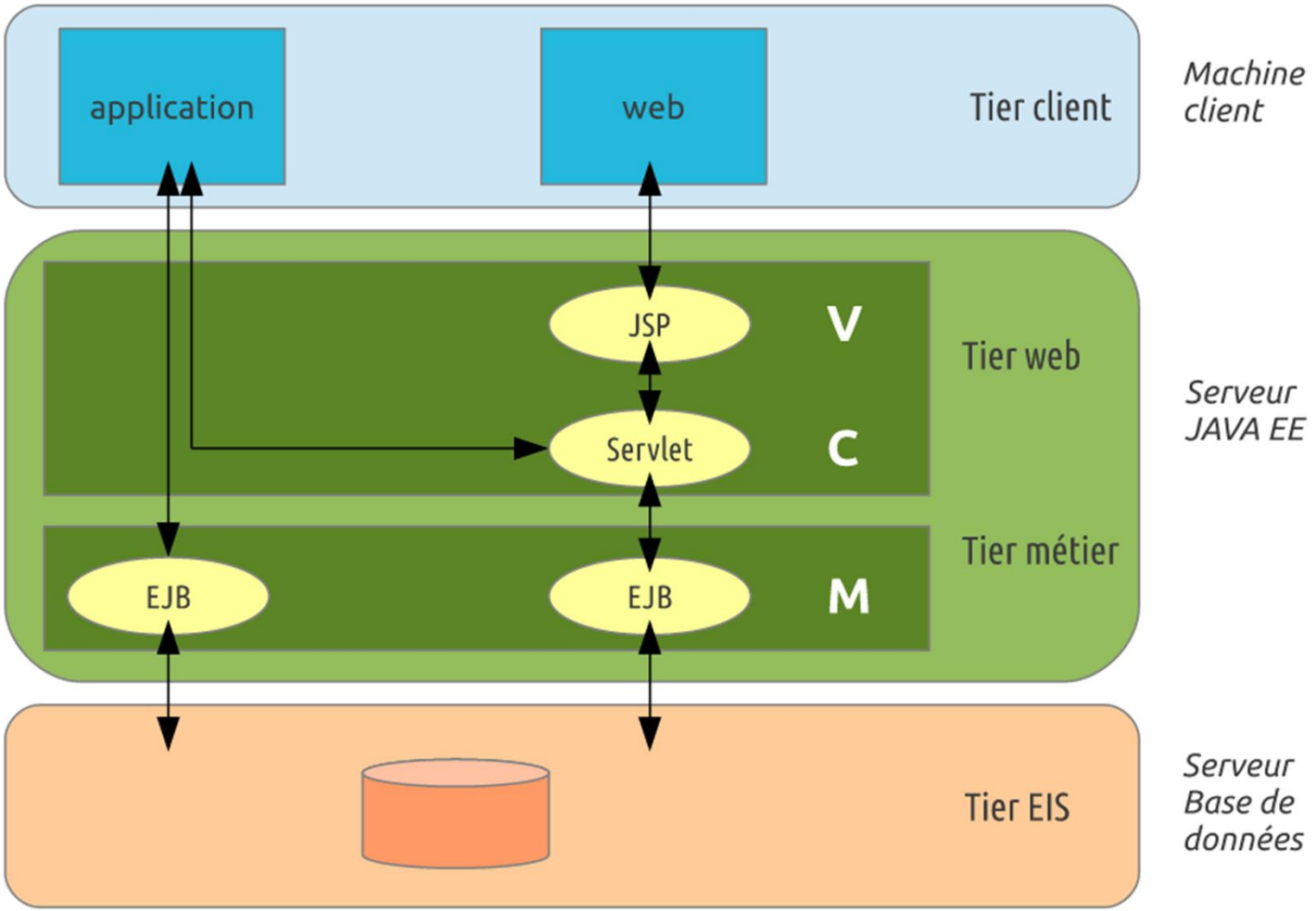
Remarque

En cas d'erreur lors du lancement d'**Apache Tomcat**

- Faire double clic sur le serveur
- Aller dans la section **Ports** et vérifier que
  - **Tomcat admin port** : 8005
  - **HTTP/1.1** : 8080
  - **AJP/1.3** : 8009

Jakarta EE

`WEB-INF` est inaccessible aux visiteurs

- Déplacer `index.html` dans `WEB-INF`
- Redémarrer le serveur
- Vérifier que `http://localhost:8080/cours-jee/` et `http://localhost:8080/cours-jee/index.html` ne sont plus accessibles.

- **JEE et modèle MVC**